Technical Requirements Document (TRD)

Project Name

TailorConnect India

Version

1.0

Technology Stack

Frontend

- Vite
- React.js
- Tailwind CSS

Backend

- Node.js
- Express.js
- REST API Architecture

Database

- MongoDB

Media Storage

- Cloudinary

Payment Gateway

- Razorpay

---

1. System Overview

TailorConnect India is a location-based marketplace platform that connects customers with nearby tailors.

The platform consists of three user roles:

1. Customer
2. Tailor
3. Admin

Core functionalities include:

- Customer registration and login
- Tailor registration and profile management
- Tailor storefront pages
- Tailor search and filtering
- Ratings and reviews
- Portfolio management
- Premium subscription purchase
- Admin management portal

---

2. High Level Architecture

Frontend Layer

React + Vite application responsible for:

- Authentication UI
- Search pages
- Tailor listing pages
- Tailor profile pages
- Review system
- Dashboard pages
- Admin panel

Backend Layer

Node.js + Express REST APIs responsible for:

- Authentication
- Business logic
- User management
- Search operations
- Review management
- Subscription management
- Razorpay integration
- Cloudinary integration

Database Layer

MongoDB stores:

- Users
- Tailors
- Reviews
- Locations
- Gallery Images
- Subscription Data
- Inquiries

Media Layer

Cloudinary stores:

- User profile photos
- Tailor profile photos
- Shop photos
- Gallery images
- Review photos

---

3. User Roles

Customer

Permissions:

- Register account
- Login
- Manage profile
- Search tailors
- View tailor profiles
- Save favorites
- Submit reviews
- Upload review photos

Tailor

Permissions:

- Register shop

- Manage profile
- Upload portfolio
- Manage gallery
- View inquiries
- Respond to reviews
- Purchase premium subscription

Admin

Permissions:

- Approve tailor accounts
- Manage users
- Manage reviews
- Manage locations
- Manage subscriptions
- View analytics

---

4. Frontend Requirements

Authentication Module

Pages:

- Customer Login
- Customer Registration
- Tailor Login
- Tailor Registration
- Forgot Password

Features:

- Form validation
- Password visibility toggle
- Error handling
- Success notifications

Home Page

Components:

- Hero Section
- Search Bar
- Category Filters
- Featured Tailors
- Top Rated Tailors
- Popular Cities

Tailor Listing Page

Features:

- Search by tailor name
- Search by city
- Search by service

Filters:

- Men's Tailoring
- Women's Tailoring
- Bridal Wear
- Alteration
- Boutique
- Verified Tailor
- Top Rated
- Open Now

Sorting:

- Distance
- Rating
- Reviews Count

Tailor Profile Page

Display:

- Profile Photo
- Shop Name
- Description
- Services
- Experience
- Gallery
- Reviews
- Ratings

- Contact Information
- WhatsApp Button

Customer Dashboard

Features:

- Profile Management
- Favorites
- Reviews Submitted
- Inquiry History

Tailor Dashboard

Features:

- Profile Management
- Gallery Management
- Reviews Management
- Subscription Status
- Inquiry Management

Admin Dashboard

Modules:

- User Management
- Tailor Verification
- Review Moderation
- Location Management
- Subscription Management
- Analytics

---

5. Backend Requirements

Authentication Service

Responsibilities:

- Registration
- Login
- Password Hashing
- JWT Generation
- Authorization

API Structure:

POST /api/auth/register

POST /api/auth/login

POST /api/auth/logout

GET /api/auth/profile

PUT /api/auth/profile

User Management Service

Responsibilities:

- Create User
- Update User
- Delete User
- Fetch User Details

Endpoints:

GET /api/users

GET /api/users/:id

PUT /api/users/:id

DELETE /api/users/:id

Tailor Management Service

Responsibilities:

- Create Tailor Profile
- Update Tailor Profile

- Manage Services
- Manage Service Radius

Endpoints:

POST /api/tailors

GET /api/tailors

GET /api/tailors/:id

PUT /api/tailors/:id

DELETE /api/tailors/:id

Search Service

Responsibilities:

- Search Tailors
- Filter Tailors
- Sort Tailors

Endpoint:

GET /api/search

Query Parameters:

- city
- district
- state
- service
- rating
- verified

Review Service

Responsibilities:

- Create Review
- Edit Review
- Delete Review
- Reply to Review

Endpoints:

POST /api/reviews

GET /api/reviews/:tailorId

PUT /api/reviews/:id

DELETE /api/reviews/:id

Inquiry Service

Responsibilities:

- Customer Inquiry Submission
- Tailor Inquiry Retrieval

Endpoints:

POST /api/inquiries

GET /api/inquiries

Subscription Service

Responsibilities:

- Premium Purchase
- Subscription Renewal
- Subscription Status

Endpoints:

POST /api/subscriptions/create-order

POST /api/subscriptions/verify-payment

GET /api/subscriptions/status

---

6. Database Design

Users Collection

Fields:

- _id
- role
- fullName
- email
- mobile
- password
- state
- district
- city
- profileImage
- favorites
- createdAt
- updatedAt

Tailors Collection

Fields:

- _id
- ownerId
- shopName
- description
- whatsappNumber
- mobileNumber
- address
- state
- district
- city
- experience
- services
- serviceRadius
- profileImage
- galleryImages
- averageRating
- reviewCount
- verified
- subscriptionType
- createdAt
- updatedAt

Reviews Collection

Fields:

- _id
- customerId
- tailorId
- rating
- reviewText
- feedbackTags
- photos
- tailorReply
- createdAt

Inquiries Collection

Fields:

- _id
- customerId
- tailorId
- customerName
- customerEmail
- customerMobile
- message
- createdAt

Subscriptions Collection

Fields:

- _id
- tailorId
- razorpayOrderId
- paymentId
- amount
- plan
- status
- startDate
- expiryDate

Locations Collection

Fields:

- _id
- state
- district
- city

---

7. Cloudinary Integration

Supported Uploads:

- User Profile Images
- Tailor Profile Images
- Shop Images
- Gallery Images
- Review Images

Folder Structure:

users/

tailors/profile/

tailors/gallery/

reviews/

Storage Flow:

Frontend → Backend API → Cloudinary → MongoDB URL Storage

---

8. Razorpay Integration

Premium Subscription Flow

Step 1:
Tailor selects premium plan.

Step 2:
Backend creates Razorpay order.

Step 3:
Frontend opens Razorpay checkout.

Step 4:
Payment completed.

Step 5:
Backend verifies payment signature.

Step 6:
Subscription activated.

Database Updated:

- Payment Status
- Plan Type
- Expiry Date

---

9. Security Requirements

Authentication

- JWT Authentication
- Protected Routes
- Role Based Access Control

Password Security

- Password Hashing

API Security

- Request Validation
- Input Sanitization
- Error Handling

Upload Security

- Image Type Validation
- Upload Size Limits
- Cloudinary Secure Upload

---

10. Performance Requirements

Frontend:

- Lazy Loading Components
- Optimized Image Rendering
- Pagination

Backend:

- Pagination APIs
- Database Query Optimization

Database:

- Indexing on:
    - city
    - district
    - state
    - rating
    - verified

---

11. MVP Scope

Included in Version 1:

- Customer Registration
- Tailor Registration
- Login System
- Tailor Storefront
- Search Tailors
- Filter Tailors
- Reviews and Ratings
- Favorites
- Cloudinary Image Upload
- Razorpay Premium Subscription
- Admin Dashboard
- Responsive UI

---

12. Deployment Structure

Frontend

React + Vite Application

Backend

Node.js + Express REST APIs

Database

MongoDB

Storage

Cloudinary

Payment

Razorpay

Communication Flow

Frontend → REST API → MongoDB

Frontend → REST API → Cloudinary

Frontend → REST API → Razorpay

Razorpay → Backend Verification → MongoDB